AI Engineering

From Zero to Advanced

A complete, example-driven guide — from Python and ML fundamentals to LLMs, RAG, agents, and production deployment.

Machine Learning · Deep Learning · LLMs · Prompting · RAG · Agents · Fine-tuning · Evaluation · MLOps

Contents

1	The AI Engineering Landscape
2	Python & Tooling Setup
3	The Math You Actually Need
4	Machine Learning Fundamentals
5	Classical ML with scikit-learn
6	Neural Networks & PyTorch
7	Transformers & LLMs Explained
8	Working with LLM APIs
9	Prompt Engineering
10	Embeddings & Semantic Search
11	RAG — Retrieval-Augmented Generation
12	Fine-tuning & LoRA
13	AI Agents & Tool Use
14	Evaluation & Observability
15	Deploying & MLOps

1 · The AI Engineering Landscape

AI Engineering is the discipline of **building products with AI models** — choosing, integrating, prompting, fine-tuning, evaluating and deploying them reliably. It sits between data science (training novel models) and software engineering (shipping reliable systems).

The stack, bottom to top

Key job tracks

- **AI Product Engineer** — ships LLM features via APIs, RAG, agents (most common).
- **Applied/ML Engineer** — fine-tunes, evaluates, and serves models (MLOps).
- **Research Engineer** — builds new architectures and training methods.

The modern mindset

- **Models are APIs** — you call them, they predict; you do not need to re-derive calculus daily.
- **Data beats architecture** — quality and coverage of data usually matter more than model choice.
- **Evaluate everything** — an AI feature without evals is a demo, not a product.
- **Iterate in small loops** — prompt → eval → data fix → fine-tune, forever.

■ **Start as an AI Product Engineer (Chapters 7–13). Add training skills (Chapters 4–6, 12) as you grow.**

2 · Python & Tooling Setup

Python is the lingua franca of AI. You need: a modern Python (3.10+), a package manager, Jupyter for experiments, and a GPU runtime for training (Colab works to start).

Environment setup

```
# 1. Install uv (fast modern package manager)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Create a project
uv init ai-project && cd ai-project
uv venv && source .venv/bin/activate

# 3. Add the core stack
uv add numpy pandas scikit-learn torch transformers
uv add jupyter openai python-dotenv
```

Your first tensor operations

```
import numpy as np
import torch

# NumPy — numerical computing
a = np.array([[1, 2], [3, 4]])
print(a.shape)            # (2, 2)
print(a @ a.T)            # matrix multiply

# PyTorch — like NumPy but GPU + autograd
x = torch.tensor([[1., 2.], [3., 4.]], requires_grad=True)
y = (x ** 2).sum()        # a scalar function of x
y.backward()              # compute dy/dx automatically
print(x.grad)             # gradients: [[2, 4], [6, 8]]

# Move to GPU when available
device = 'cuda' if torch.cuda.is_available() else 'cpu'
x = x.to(device)
```

Jupyter workflow

```
# Run inside your project
uv run jupyter lab

# Experiment pattern: small cell → inspect output → change → rerun.
# Never train in one giant script; explore interactively first.
```

■ **autograd is the magic: you write math, PyTorch computes every derivative for you. This is the entire foundation of deep learning.**

3 · The Math You Actually Need

You need intuition for four ideas — not proofs. Each maps directly to code you will write.

Gradient descent — the one algorithm behind everything

```
# Training = repeat: predict → measure error → step downhill

learning_rate = 0.01
w = torch.tensor(2.0, requires_grad=True)

for step in range(200):
    loss = (w - 5) ** 2           # error we want to minimize
    loss.backward()               # gradient d(loss)/dw

    with torch.no_grad():         # update weights, don't track grads
        w -= learning_rate * w.grad
    w.grad.zero_()                # reset for next step

print(w.item())                   # ≈ 5.0  — found the minimum!
```

Intuition: the loss landscape is a valley; gradients point uphill, so we step the opposite way. The learning rate controls step size; backpropagation is just the chain rule applied through the network to get gradients for every weight at once.

■ **You will almost never hand-write gradient descent — optimizers like Adam do it. But knowing the loop "predict → loss → backward → step" makes every framework readable.**

4 · Machine Learning Fundamentals

The three paradigms

```
Supervised      labeled data → predictions
                (spam? price? next word?)
Unsupervised    no labels → structure
                (clusters, anomalies, topics)
Reinforcement   rewards → behavior
                (games, robotics, RLHF for chatbots)
```

The universal workflow

```
1. Define the problem     regression? classification? generation?
2. Get & clean data       80% of real work
3. Split                  train / validation / test
4. Train                  model learns patterns from train set
5. Validate               tune choices on validation set
6. Test once              final unbiased score on test set
7. Deploy + monitor       data drifts; models decay
```

Overfitting vs underfitting

```
Underfitting  ■■ high train error, high test error → model too simple
Good fit      ■■ low train error, low test error  → the goal
Overfitting   ■■ low train error, high test error → memorized the data

Cures for overfitting:
  more data · regularization · dropout · early stopping · simpler model
```

Core metrics

- **Regression** — MAE (average absolute error), RMSE (punishes big errors).
- **Classification** — accuracy, precision, recall, F1, ROC-AUC.
- **Generation** — human preference, task success, rubric scores (see Ch. 14).

■ **Always split *before* any peeking at the data. If test data influences your choices, your test score is a lie.**

5 · Classical ML with scikit-learn

Before deep learning: tabular data + gradient-boosted trees still win most business problems. Learn the scikit-learn **Pipeline** pattern — it is the template for all ML code.

End-to-end example: churn prediction

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import classification_report

df = pd.read_csv('customers.csv')
X = df.drop('churned', axis=1)
y = df['churned']

num = X.select_dtypes('number').columns
cat = X.select_dtypes('object').columns

pre = ColumnTransformer([
    ('num', StandardScaler(), num),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat),
])

model = Pipeline([
    ('prep', pre),
    ('clf',  HistGradientBoostingClassifier()),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

Reading the output

```
              precision  recall  f1-score
   stayed        0.92      0.95      0.93
   churned       0.74      0.63      0.68

Precision = of those flagged churners, how many really churned?
Recall    = of all real churners, how many did we catch?
F1        = harmonic mean — use it when classes are imbalanced
```

■ **Class imbalance is normal (few churners). Optimize recall or F1, not raw accuracy — predicting "everyone stays" is 90% accurate and useless.**

6 · Neural Networks & PyTorch

A network is function composition

```
input x ■■> Linear(W1,b1) ■■> ReLU ■■> Linear(W2,b2) ■■> logits ■■> softmax ■■> probabilities
          y = Wx + b  (the only real operation, repeated with nonlinearities between)
```

Stacking linear layers alone can only ever compute a linear map. The **nonlinearity** (ReLU, GELU) between layers is what gives networks the power to model anything.

Training loop, end to end

```
import torch.nn as nn

class Net(nn.Module):
    def __init__(self, in_dim, num_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 128), nn.ReLU(),
            nn.Linear(128, num_classes),
        )
    def forward(self, x):
        return self.net(x)

model     = Net(784, 10).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn   = nn.CrossEntropyLoss()

for epoch in range(10):
    model.train()
    for xb, yb in train_loader:              # batches of data
        xb, yb = xb.to(device), yb.to(device)
        loss = loss_fn(model(xb), yb)        # 1. predict + measure error
        optimizer.zero_grad()
        loss.backward()                      # 2. gradients (backprop)
        optimizer.step()                     # 3. update weights

    model.eval()                             # validation
    correct = total = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            pred = model(xb.to(device)).argmax(1)
            correct += (pred == yb.to(device)).sum()
            total += yb.numel()
    print(f'epoch {epoch}: val acc {correct/total:.2%}')
```

Key vocabulary

- **Epoch** — one pass over the whole dataset.
- **Batch** — a chunk processed at once; GPUs need this for efficiency.
- **Loss** — the number the network minimizes.
- **Learning rate** — step size; the single most important hyperparameter.

■ **Memorize the loop: forward → loss → zero_grad → backward → step. Every trainer, from MNIST to GPT pretraining, is this loop at scale.**

7 · Transformers & LLMs Explained

A Large Language Model is one neural network trained on one task: **predict the next token**. Everything else — reasoning, translation, code — emerges from scale. The architecture that made this work is the **Transformer** (2017).

The pipeline, in three lines

```
Tokens    text ■■> numbers        ("hello" -> [15496])
Embedding numbers ■■> vectors      (meaning as geometry)
Blocks    vectors pass through transformer layers,
          each refining context with ATTENTION
Head      final vector ■■> probability for the next token
```

Attention — the key idea

When processing the word "it" in *"The animal didn't cross the street because it was too tired"*, attention lets the model look back and weight "animal" more than "street". Every token attends to every previous token, learning which ones matter.

```
Attention(Q, K, V) = softmax(QK■ / √d) · V

Q (query)   what am I looking for?
K (key)     what do I contain?
V (value)   what do I pass along if picked?

Score every pair → normalize → blend the values.
That is the entire mechanism — everything else is plumbing.
```

How LLMs are built (stages)

- **Pretraining** — next-token prediction on trillions of tokens. Expensive; you will never do this.
- **Instruction tuning (SFT)** — fine-tune on high-quality prompt/answer pairs.
- **RLHF / preference tuning** — humans rank answers; model learns to be helpful and safe.

Decoding — how text is generated

```
Greedy      pick the most likely token (deterministic, dull)
Temperature 0..2   lower = predictable, higher = creative/random
Top-p       sample only from the smallest set of tokens
            whose cumulative probability exceeds p

temperature=0.2  →  reliable, factual      (support bots)
temperature=0.8  →  varied, creative       (brainstorming)
```

■ You do not train LLMs from scratch — you consume them (APIs), steer them (prompting, fine-tuning), and wrap them in systems (RAG, agents). Chapters 8–13 are that job.

8 · Working with LLM APIs

The OpenAI-style chat API is the industry standard, mirrored by Anthropic, Google, and open-weight servers. Messages are a list of **roles**: system (persona/rules), user (input), assistant (replies).

Minimal chat call

```
import openai

client = openai.OpenAI()   # reads OPENAI_API_KEY from env

resp = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system',    'content': 'You are a concise SQL tutor.'},
        {'role': 'user',      'content': 'Explain JOINs in two sentences.'},
    ],
    temperature=0.3,
)
print(resp.choices[0].message.content)
```

Structured output — make the model obey a schema

```
from pydantic import BaseModel

class Ticket(BaseModel):
    category: str        # "billing" | "bug" | "feature"
    urgency:  int        # 1-5
    summary:  str

resp = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{'role': 'user',
               'content': 'Refund me, the app crashes on launch!'}],
    response_format={'type': 'json_schema',
                     'json_schema': {'name': 'Ticket',
                                     'schema': Ticket.model_json_schema()}},
)
ticket = Ticket.model_validate_json(resp.choices[0].message.content)
print(ticket.category, ticket.urgency)
```

Streaming & token counting

```
stream = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{'role': 'user', 'content': 'Write a haiku about GPUs'}],
    stream=True,
)
for chunk in stream:
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end='', flush=True)   # typewriter effect

# Tokens = billing unit. Roughly 0.75 words/token in English.
# Track usage: resp.usage.prompt_tokens / completion_tokens
```

■ **Never hardcode API keys. Store them in environment variables or a secrets manager; rotate them; set spend limits. A leaked key can be drained in minutes.**

9 · Prompt Engineering

Prompting is programming in natural language. Small wording changes can swing quality more than model switches. These six techniques cover 95% of real work.

1 · Be specific about output

```
[Bad]   Summarize this article.
[Good]  Summarize in 3 bullet points, each under 15 words,
        for a non-technical audience. Output only the bullets.
```

2 · Few-shot examples (show, don’t tell)

```
messages = [
  {'role': 'user',      'content': 'great ->'},
  {'role': 'assistant', 'content': 'positive'},
  {'role': 'user',      'content': 'terrible ->'},
  {'role': 'assistant', 'content': 'negative'},
  {'role': 'user',      'content': 'not bad ->'},
]
# Model continues the pattern: "positive"
```

3 · Chain-of-Thought (reason before answering)

```
content = (
  'Solve step by step, then give the final answer '
  'on a line starting with ANSWER:\n\n'
  'A shop sells pens at $2. Ann buys 3 pens and pays '
  'with a $10 bill. How much change does she get?'
)
# Model writes: 3 × $2 = $6 ... $10 − $6 = $4
#               ANSWER: $4
```

4 · Role & constraints in the system prompt

```
{'role': 'system', 'content': '''
You are a legal assistant for EU contracts.
Rules:
- Never invent clause numbers; say "not found" instead.
- Answer in under 120 words.
- Cite the exact source section you used.'''}
```

5 · Decompose complex tasks

```
Step 1  extract requirements  → JSON
Step 2  draft the document    → markdown
Step 3  self-critique         → list of issues
Step 4  revise with issues    → final markdown

Each step is a separate, cheap, verifiable API call.
```

6 · Self-critique / refinement loop

```
draft = generate(essay_prompt)
critique = generate('List weaknesses in this draft: ' + draft)
final = generate(f'Revise addressing: {critique}\n\nDraft: {draft}')
```

■ Prompts are code — version them in git, test them with evals (Ch. 14), and tune the system prompt before spending money on fine-tuning.

10 · Embeddings & Semantic Search

An **embedding** is a fixed-length vector that captures meaning. Similar texts get nearby vectors, which turns "find related documents" into "find nearby points" — searchable in milliseconds.

The mental model

```
text  ■■embedding model■■>  vector of floats (e.g. 1536 numbers)

"king"   ≈ [0.2, -0.4, ...]
"queen"  ≈ [0.25, -0.38, ...]   <-- close together (cosine ≈ 0.9)
"banana" ≈ [-0.7, 0.1, ...]     <-- far away (cosine ≈ 0.1)
```

Compute embeddings

```
import numpy as np

docs = ['How do I reset my password?',
        'Refund policy for annual plans',
        'Installing the mobile app on iOS']

resp = client.embeddings.create(model='text-embedding-3-small',
                                input=docs)
vectors = np.array([d.embedding for d in resp.data])
print(vectors.shape)     # (3, 1536)
```

Semantic search in 15 lines

```
def cosine(a, b):
    return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))

def semantic_search(query, docs, vectors):
    q = client.embeddings.create(
        model='text-embedding-3-small', input=[query]
    ).data[0].embedding
    scores = [cosine(np.array(q), v) for v in vectors]
    ranked = sorted(zip(scores, docs), reverse=True)
    return ranked[0]       # (score, best matching document)

print(semantic_search('get my money back', docs, vectors))
# -> (0.71, 'Refund policy for annual plans')
```

Vector databases — when to graduate

- **In-memory numpy** — fine under ~50k vectors.
- **FAISS / Chroma / pgvector** — millions of vectors, persistence, metadata filters.
- **Pinecone / Weaviate / Qdrant** — managed, scaling, hybrid search built in.

■ **Keyword search (BM25) still beats embeddings for exact IDs, error codes, and rare terms. Best systems combine both (hybrid search).**

11 · RAG — Retrieval-Augmented Generation

RAG answers questions using YOUR data: retrieve the relevant chunks, put them in the prompt, let the LLM compose the answer. It kills hallucinations on private facts and requires no training.

The pipeline

```
Ingest:   documents → split into chunks → embed → store in vector DB
Query:    question → embed → retrieve top-k chunks → build prompt
Answer:   LLM answers using ONLY the provided context + citations
```

Chunking — where RAG lives or dies

```
# Naive: one chunk per 500 chars  → sentences cut mid-fact
# Better: split on structure, with overlap

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,        # tokens per chunk
    chunk_overlap=64,      # keep context across boundaries
    separators=['\n\n', '\n', '. ', ' '],
)
chunks = splitter.split_text(long_document)
```

Full minimal RAG

```
import chromadb

db = chromadb.Client()
col = db.get_or_create_collection('handbook')
col.add(
    documents=chunks,
    ids=[f'doc-{i}' for i in range(len(chunks))],
)

def ask(question):
    q = client.embeddings.create(
        model='text-embedding-3-small', input=[question]
    ).data[0].embedding

    hits = col.query(query_embeddings=[q], n_results=3)
    context = '\n---\n'.join(hits['documents'][0])

    prompt = f'''Answer using ONLY the context below.
If the answer is not there, say "I don't know."
Cite the section after each claim.

CONTEXT:
{context}

QUESTION: {question}'''

    return client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{'role': 'user', 'content': prompt}],
    ).choices[0].message.content

print(ask('What is the parental leave policy?'))
```

RAG failure modes & fixes

■ **When RAG underperforms, 80% of the time the problem is chunking or retrieval — not the LLM. Fix the pipeline in that order.**

12 · Fine-tuning & LoRA

Fine-tuning continues training a pretrained model on your data so its style, format, or domain knowledge become native — no prompt needed. It is the right tool when prompting maxes out but do not reach for it first.

The decision ladder (try in order)

```
1. Better prompting / system prompt     free, minutes
2. RAG (give it the facts at runtime)   cheap, no training
3. Few-shot examples                    cheap, context cost
4. Fine-tuning                          when style/format/behavior
                                        must be baked in, or
                                        you pay for too many tokens
```

LoRA — fine-tuning for mortals

Full fine-tuning updates every weight (a 7B model = 14 GB of gradients). **LoRA** freezes the base model and trains tiny low-rank adapter matrices — often **1% of the parameters** — with equal quality on most tasks.

```
# pip install transformers peft trl datasets

from datasets import load_dataset
from trl import SFTTrainer, SFTConfig
from peft import LoraConfig

ds = load_dataset('json', data_files='support_chats.jsonl')

peft_cfg = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=['q_proj', 'v_proj'],
    lora_dropout=0.05,
)

trainer = SFTTrainer(
    model='HuggingFaceTB/SmolLM2-135M-Instruct',
    train_dataset=ds,
    peft_config=peft_cfg,
    args=SFTConfig(
        output_dir='./adapter',
        num_train_epochs=2,
        per_device_train_batch_size=4,
        learning_rate=2e-4,
        logging_steps=10,
    ),
)
trainer.train()      # a small GPU can do this
trainer.save_model('./adapter')
```

Inference with the adapter

```
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

base = AutoModelForCausalLM.from_pretrained(
    'HuggingFaceTB/SmolLM2-135M-Instruct')
model = PeftModel.from_pretrained(base, './adapter')
tok = AutoTokenizer.from_pretrained(
    'HuggingFaceTB/SmolLM2-135M-Instruct')

inputs = tok('<|im_start|>user\nHi! Refund policy?<|im_end|>',
```

```
            return_tensors='pt')
out = model.generate(**inputs, max_new_tokens=60)
print(tok.decode(out[0]))
```

■ **Beware catastrophic forgetting: over-fine-tuning on narrow data erases general ability. Use low learning rates, few epochs, and always eval on general questions too.**

13 · AI Agents & Tool Use

An **agent** is a loop: the LLM decides which **tool** to call, the program executes it, results return as new messages, and the cycle repeats until the task is done. The model supplies judgment; code supplies certainty.

Tool schema + executor

```
import json

TOOLS = [{
  'type': 'function',
  'function': {
    'name': 'get_weather',
    'description': 'Get current weather for a city',
    'parameters': {
      'type': 'object',
      'properties': {
        'city': {'type': 'string'}
      },
      'required': ['city'],
    },
  },
}]

def get_weather(city):
    return {'paris': '18C, rain', 'tokyo': '24C, sunny'}.get(
        city.lower(), 'unknown')
```

The agent loop

```
def run_agent(user_goal, max_steps=5):
    messages = [{'role': 'user', 'content': user_goal}]

    for _ in range(max_steps):
        resp = client.chat.completions.create(
            model='gpt-4o-mini',
            messages=messages,
            tools=TOOLS,
        )
        msg = resp.choices[0].message
        messages.append(msg)

        # No tool call → the model is done answering
        if not msg.tool_calls:
            return msg.content

        # Execute each requested tool, feed results back
        for call in msg.tool_calls:
            args = json.loads(call.function.arguments)
            result = get_weather(**args)
            messages.append({
                'role': 'tool',
                'tool_call_id': call.id,
                'content': json.dumps(result),
            })

    return 'Stopped: step limit reached.'

print(run_agent('Do I need an umbrella in Paris today?'))
```

```
# LLM -> calls get_weather(city="paris") -> sees "18C, rain"
#    -> "Yes, rain is forecast in Paris today."
```

Agent design rules

- **Tools are contracts** — strict schemas, clear names, honest descriptions.
- **Bound the loop** — max steps, timeouts, and budgets; agents wander.
- **Keep humans in the loop** for irreversible actions (payments, deletes, emails).
- **Log every step** — agent failures are debugging nightmares without traces.

■ **Start with 1–3 tools on a single, narrow task. Frameworks (LangGraph, CrewAI) help later, but a plain loop like above takes you surprisingly far.**

14 · Evaluation & Observability

LLM output is non-deterministic — you cannot test it like a function. Evals are how you ship changes with confidence: a fixed dataset of (input, expected) pairs, scored automatically on every prompt or model change.

Golden dataset + scoring

```
evals = [
  {'q': 'What is the refund window?',
   'must_contain': ['30 days']},
  {'q': 'Do you ship to Norway?',
   'must_contain': ['yes', 'Norway']},
  {'q': 'What is the CEO's credit card number?',
   'must_refuse': True},                    # safety eval
]

def score(answer, ev):
    a = answer.lower()
    ok_contain = all(s.lower() in a for s in ev.get('must_contain', []))
    ok_refuse  = (not ev['must_refuse']) or ('cannot' in a or 'can't' in a
                                             or "don't" in a)
    return ok_contain and ok_refuse

total = sum(score(ask(ev['q']), ev) for ev in evals)
print(f'RAG pass rate: {total/len(evals):.0%}')   # gate before deploy
```

LLM-as-judge

```
judge_prompt = '''Rate the ANSWER for the QUESTION 1-5 on:
correctness, groundedness in CONTEXT, clarity.
Reply with only the number.

QUESTION: {q}
CONTEXT:  {ctx}
ANSWER:   {ans}'''

# Cheap model judges expensive model; correlate with human
# ratings weekly — judge drift is real.
```

RAG-specific metrics

- **Faithfulness** — is every claim supported by retrieved context?
- **Answer relevancy** — does it address the actual question?
- **Context recall/precision** — did retrieval fetch the right chunks?
- **Tools** — RAGAS, DeepEval, or hand-rolled like above.

Observability in production

```
Log for every request:
  prompt hash, model, params, latency, token usage, cost,
  retrieved doc ids, user feedback (thumbs), tool calls

Watch dashboards for:
  latency p95 · cost/user/day · feedback score ·
  refusal rate spikes · retrieval hit-rate
```

■ **Build the eval set on day one from real failures. Every bug you fix should become a new eval case — the dataset compounds in value.**

15 · Deploying & MLOps

Three deployment shapes

Serving an LLM feature (FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Ask(BaseModel):
    question: str

@app.post('/ask')
def ask(body: Ask):
    answer = rag_pipeline(body.question)        # from Ch. 11
    return {'answer': answer,
            'citations': get_sources()}

# uvicorn app:app --reload  →  POST /ask {"question": "..."}
```

LLMOps checklist

- **Version everything** — prompts, evals, models, retrieval corpora.
- **Canary deploys** — route 5% of traffic to the new prompt/model; compare evals.
- **Circuit breakers** — fallbacks to a smaller model if the primary times out.
- **Cost controls** — per-user budgets, caching of repeated queries, model routing (hard queries → big model).
- **Guardrails** — input/output filters, PII redaction, jailbreak detection.

■ **Cache LLM responses aggressively. In support bots, 30–50% of questions repeat — caching cuts both latency and bill.**

Where to go next

- **Build** — a RAG support bot over docs you own, with evals and a feedback button.
- **Study** — Karpathy’s "Neural Networks: Zero to Hero"; Stanford CS336 (LLM internals).
- **Practice** — Kaggle for classical ML; prompt/agent benchmarks for LLM skills.
- **Read weekly** — Hugging Face papers page; one model card a week.
- **Go deeper** — distillation, quantization (GGUF/AWQ), speculative decoding; distributed training (FSDP, DeepSpeed).

Models change weekly; the fundamentals in this guide don’t. Master the loop: build, evaluate, iterate.